

PARERE GRC

Sistema de Gestión de Riesgo y Proveedores

Documento de Especificación de Servicios

Registro de Proveedores · Validación Final y Cambio de Estado · Firma Digital

Control de versiones

Fecha	Versión	Descripción	Autor
23-08-2026	1.0	Especificación inicial de los servicios de Registro de Proveedores, Validación Final / Cambio de Estado y Firma Digital del sistema Parere GRC.	Adriana Barrios / Juan Cadena

Aprobaciones

Nombre / Rol	Fecha	Firma
Analista / Desarrollador		
Analista / Desarrollador		
Instructor SENA		
Comité de Proyecto ADSO		

Tabla de Contenido

Tabla de Contenido	2
1. Breve Descripción	4
2. Versión del Servicio.....	4
3. Dueños del Servicio.....	4
4. Descripción de la Interfaz de los Servicios	4
5. Detalle de las Funciones del Servicio	6
5.1 Registro de Proveedores.....	6
Información de entrada	6
Ejemplo REQUEST	6
Información de salida	8
Ejemplo RESPONSE	8
Restricciones en el servicio	8
5.2 Validación Final y Cambio de Estado del Proveedor.....	9
5.2.1 create — POST /validacion-final	9
Ejemplo REQUEST	9
Ejemplo RESPONSE	9
5.2.2 getAll — GET /validacion-final	9
5.2.3 updateEstado — PATCH /proveedores/{id}/estado	10
Ejemplo REQUEST	10
Ejemplo RESPONSE	10
Restricciones en el servicio	10
5.3 Firma Digital.....	11
5.3.1 solicitarFirma — POST /firmas/solicitar	11
Ejemplo RESPONSE	11
5.3.2 validarToken — GET /firmas/validar.....	11
Ejemplo RESPONSE	11
5.3.3 firmarFormulario — POST /firmas/firmar.....	11
Ejemplo RESPONSE	11
Restricciones en el servicio	11
6. Manejo de Requerimientos No Funcionales.....	13
7. Manejo de Requerimientos Funcionales	14
7.1 Configuración en base de datos.....	14
7.1.1 Tabla proveedor.....	14
7.1.2 Tabla validacion	14
7.1.3 Tabla validacion_final (gestionada por Hibernate).....	15

7.1.4 Tabla estado_proveedor	15
7.1.5 Tabla campo_validacion	15
7.1.6 Tabla firma_token (gestionada por Hibernate)	16
7.2 Llamadas del servicio	16
7.3 Manejo de errores	16

1. Breve Descripción

Este documento describe la especificación funcional y técnica de tres servicios REST del backend de Parere GRC, expuestos mediante Spring Boot, que sostienen el ciclo de vida ético de un proveedor: (1) el registro completo del proveedor y su información asociada, (2) la validación final y el cambio de estado del proveedor por parte de los roles evaluadores, y (3) la firma digital del formulario de vinculación.

Los tres servicios comparten la misma base de datos relacional (esquema parere_grc, MySQL) y el mismo formato de respuesta envolvente `ResponseDto<T> { "data": ... }`.

2. Versión del Servicio

1.0

3. Dueños del Servicio

De acuerdo con una matriz RACI (Responsible, Accountable, Consulted, Informed) se indican las áreas o personas responsables de los servicios:

Responsable	Adriana Gineth Barrios Aponte — Desarrolladora / Analista
Responsable	Juan Carlos Cadena Muñoz — Desarrollador / Analista
Accountable	Instructor(a) del programa de formación ADSO — SENA
Consultado	Comprador, Analista de Riesgo y Oficial de Cumplimiento (usuarios funcionales de los servicios)
Informado	Proveedor final (usuario externo que diligencia y firma el registro)

4. Descripción de la Interfaz de los Servicios

En esta sección se describen las funciones u operaciones expuestas por el backend:

url base: `http://localhost:8080` (Spring Boot, sin prefijo `/api`)

Servicio	Descripción	Endpoint	Función	Método
Registro de Proveedores	Recibe y persiste en una sola petición la información de la empresa, contactos, representantes, socios, datos bancarios, financieros y de LAFT/SARLAFT.	<code>/proveedores/registro-completo</code>	<code>registroCompleto</code>	POST
Validación Final	Registra la decisión consolidada de un rol evaluador (Compras, Riesgos u Oficial de Cumplimiento) sobre un proveedor, enlazando las validaciones atómicas asociadas.	<code>/validacion-final</code>	<code>create</code>	POST

Servicio	Descripción	Endpoint	Función	Método
Validación Final	Consulta el histórico de decisiones de validación final registradas para todos los proveedores.	/validacion-final	getAll	GET
Cambio de Estado	Actualiza de forma parcial y seguido de la validación final, el estado del ciclo de vida del proveedor.	/proveedores/{id}/estado	updateEstado	PATCH
Firma Digital	Genera un token único de firma para un proveedor y construye el enlace que se envía al cliente.	/firmas/solicitar	solicitarFirma	POST
Firma Digital	Valida que un token de firma exista, no haya sido utilizado y no esté expirado antes de mostrar el formulario.	/firmas/validar	validarToken	GET
Firma Digital	Registra la aceptación digital del formulario, marca el token como utilizado y actualiza el estado del proveedor.	/firmas/firmar	firmarFormulario	POST

5. Detalle de las Funciones del Servicio

En esta sección se describe el detalle de las funciones y las reglas que aplican para cada uno de los servicios, incluyendo los mensajes de entrada y salida.

5.1 Registro de Proveedores

Función que permite registrar o actualizar en una sola petición transaccional al proveedor junto con sus contactos, representantes legales, socios, información bancaria, financiera y de debida diligencia (LAFT).

Información de entrada

Tag / Atributo	Descripción	Tipo	Observaciones
idUsuarioAsignado	Identificador del usuario del portal de autogestión que diligencia el registro.	String	Si no llega o no es numérico, se usa el usuario 1 por defecto.
empresa	Datos generales de la empresa o persona natural proveedora.	Objeto	nombre, numeroIdentificacion y correo son la llave de búsqueda para actualizar un registro existente.
contactos	Personas de contacto de la empresa.	Arreglo de Objeto	Reemplaza por completo los contactos previamente registrados.
representantes	Representantes legales (principal y suplentes).	Arreglo de Objeto	Reemplaza por completo los representantes previamente registrados.
socios	Socios o accionistas de la empresa.	Arreglo de Objeto	Reemplaza por completo los socios previamente registrados.
validaciones	Resultado de validaciones cruzadas (web vs. documento) por campo.	Arreglo de Objeto	Reemplaza las validaciones previas del proveedor.
bancaria	Cuenta bancaria y método de pago del proveedor.	Objeto	Opcional.
financiera	Cifras financieras (activos, pasivos, patrimonio, ingresos, gastos).	Objeto	Opcional.
laft	Preguntas de debida diligencia SARLAFT/LAFT (p1..p5).	Objeto	Opcional, valores booleanos.

Ejemplo REQUEST

```
{
  "idUsuarioAsignado": "12",
  "empresa": {
    "nombre": "Suministros Andinos SAS",
    "idTipoPersona": 2,
    "categoria": "Proveedor de insumos de oficina",
    "idTipoIdentificacion": 2,
    "numeroIdentificacion": "900123456",
    "ciiu": "4649",
```

```
"paginaWeb": "https://suministrosandinos.com",
"idTipoTelefono": 1,
"telefono": "6013456789",
"correo": "contacto@suministrosandinos.com",
"idPais": 1,
"idDepartamento": 14,
"idMunicipio": 149,
"direccion": "Calle 45 # 12-30, Bogotá D.C."
},
"contactos": [
  {
    "nombres": "Laura",
    "apellidos": "Gómez Pérez",
    "cargo": "Gerente Comercial",
    "idTipoIdentificacion": 1,
    "numeroIdentificacion": "1020304050",
    "idTipoTelefono": 1,
    "telefono": "3101234567",
    "correo": "laura.gomez@suministrosandinos.com"
  }
],
"representantes": [
  {
    "esPrincipal": true,
    "nombres": "Carlos",
    "apellidos": "Ramírez Díaz",
    "cargo": "Representante Legal",
    "idTipoIdentificacion": 1,
    "numeroIdentificacion": "79456123",
    "nacionalidad": "Colombiana",
    "idTipoTelefono": 1,
    "telefono": "3009876543",
    "correo": "carlos.ramirez@suministrosandinos.com"
  }
],
"socios": [
  {
    "idTipoPersona": 1,
    "nombreCompleto": "Ana María Torres",
    "idTipoIdentificacion": 1,
    "numeroIdentificacion": "52741963",
    "participacion": 60,
    "nacionalidad": "Colombiana"
  }
],
"validaciones": [
  {
    "idCampoValidacion": 3,
    "valorWeb": "900123456-7",
    "valorDocumento": "900123456-7",
    "resultadoValidacion": true,
    "comentarios": "El NIT coincide con el certificado de Cámara de Comercio"
  }
],
"bancaria": {
  "tipoCuenta": "AHORROS",
  "numeroCuenta": "21000456789",
  "banco": "Bancolombia",
  "idMetodoPago": 2
},
"financiera": {
```

```

    "activos": 250000000,
    "pasivos": 80000000,
    "patrimonio": 170000000,
    "totalIngresos": 420000000,
    "totalGastos": 310000000
  },
  "laft": {
    "p1": true,
    "p2": true,
    "p3": false,
    "p4": true,
    "p5": true
  }
}

```

Información de salida

Tag / Atributo	Descripción	Tipo
data	true si el registro (creación o actualización) fue exitoso.	Boolean

Ejemplo RESPONSE

```

{
  "data": true
}

```

Restricciones en el servicio

- Si ya existe un proveedor con el idUsuarioAsignado recibido (o, en su defecto, con el correo de la empresa) se actualiza ese registro en lugar de crear uno nuevo.
- Los identificadores de catálogo (tipo de persona, tipo de identificación, tipo de teléfono, municipio, método de pago) se asumen en 1 cuando no llegan en la petición.
- Contactos, representantes y socios se reemplazan por completo en cada envío: los registros previos asociados al proveedor se eliminan antes de insertar los nuevos.
- La operación es transaccional (@Transactional): si falla la persistencia de cualquiera de las entidades relacionadas, se revierte toda la operación.
- El proveedor nuevo queda con id_estado_proveedor = 4 (valor por defecto de la tabla proveedor) hasta que avance en el flujo de validación.

5.2 Validación Final y Cambio de Estado del Proveedor

Conjunto de funciones que permiten a un rol evaluador (Comprador, Analista de Riesgo u Oficial de Cumplimiento) registrar su decisión consolidada sobre un proveedor y reflejarla en el estado del ciclo de vida del proveedor. El flujo se ejecuta en dos llamadas consecutivas desde el Frontend: primero se crea la validación final y luego se actualiza el estado del proveedor.

5.2.1 create — POST /validacion-final

Tag / Atributo	Descripción	Tipo	Obligatorio
idProveedor	Identificador del proveedor evaluado.	Integer	Sí
comentarioFinal	Observación consolidada del evaluador.	String	No
estadoValidacion	Código de decisión (se usa como id de estado del proveedor, ej. "10" o "11").	String	Sí
creadoPor	Identificador del usuario que registra la decisión.	Integer	Sí
validationIds	Identificadores de las validaciones atómicas (tabla validacion) que sustentan la decisión.	Arreglo de Integer	No

Ejemplo REQUEST

```
{
  "idProveedor": 34,
  "comentarioFinal": "Se revisaron los soportes financieros y jurídicos. Sin novedades relevantes.",
  "estadoValidacion": "11",
  "creadoPor": 8,
  "validationIds": [
    101,
    102,
    103
  ]
}
```

Ejemplo RESPONSE

```
{
  "data": true
}
```

5.2.2 getAll — GET /validacion-final

Retorna el histórico completo de decisiones (ValidacionFinalEntity) registradas para todos los proveedores, incluyendo las validaciones atómicas enlazadas.

```
{
  "data": [
    {
      "idValidacionFinal": 57,
      "idProveedor": 34,
      "comentarioFinal": "Sin novedades relevantes.",

```

```

    "estadoValidacion": "11",
    "creadoPor": 8,
    "fechaCreado": "2026-08-20T09:15:00",
    "activo": true
  }
]
}

```

5.2.3 updateEstado — PATCH /proveedores/{id}/estado

Actualiza de forma parcial el estado del ciclo de vida del proveedor. Se invoca inmediatamente después de create para reflejar la decisión registrada en validacion-final.

Tag / Atributo	Descripción	Tipo
idEstadoProveedor	Nuevo identificador de estado a aplicar (tabla estado_proveedor).	Integer
modificadoPor	Identificador del usuario que ejecuta el cambio.	Integer

Ejemplo REQUEST

```

{
  "idEstadoProveedor": 11,
  "modificadoPor": 8
}

```

Ejemplo RESPONSE

```

{
  "data": true
}

```

Restricciones en el servicio

- El campo estadoValidacion se interpreta como el id numérico de estado a asignar al proveedor; si no es numérico, se usa 11 (revisado sin novedad) por defecto, salvo el texto "REVISADO_CON_NOVEDAD", que asigna 10.
- Las validaciones atómicas referenciadas en validationIds quedan enlazadas a la validación final mediante el campo id_validacion_final en la tabla validacion.
- Si se envía idEstadoProveedor = 11 en el PATCH, se registra automáticamente fecha_aprobacion con la fecha y hora actuales.
- El servicio no valida que idProveedor exista antes de crear la validación final; si el proveedor no existe, la actualización de estado (paso 2) se omite silenciosamente.
- No existe endpoint de edición ni borrado de una validación final: el histórico es append-only.

5.3 Firma Digital

Conjunto de funciones que permiten generar un enlace de firma para un proveedor, validar su vigencia y registrar la aceptación digital del formulario de vinculación.

5.3.1 solicitarFirma — POST /firmas/solicitar

Tag / Atributo	Descripción	Tipo	Ubicación
idProveedor	Identificador del proveedor al que se le solicita la firma.	Integer	Query param

Ejemplo RESPONSE

```
{
  "data": {
    "token": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "fechaExpiracion": "2026-08-24T10:15:30",
    "linkFirma": "/Frontend/sheets/sign_form.html?token=3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

5.3.2 validarToken — GET /firmas/validar

Tag / Atributo	Descripción	Tipo	Ubicación
token	Token UUID a validar.	String	Query param

Ejemplo RESPONSE

```
{
  "data": {
    "idToken": 15,
    "idProveedor": 34,
    "token": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "fechaExpiracion": "2026-08-24T10:15:30",
    "utilizado": false,
    "fechaFirmado": null,
    "ipFirma": null
  }
}
```

5.3.3 firmarFormulario — POST /firmas/firmar

Tag / Atributo	Descripción	Tipo	Ubicación
token	Token UUID que se está confirmando.	String	Query param

Ejemplo RESPONSE

```
{
  "data": true
}
```

Restricciones en el servicio

- El token se genera como un UUID aleatorio y expira 24 horas después de su creación.
- Al solicitar una nueva firma para un proveedor, cualquier token anterior no utilizado de ese mismo proveedor se marca como utilizado (queda invalidado).
- `validarToken` lanza error si el token no existe, ya fue utilizado, o si la fecha actual es posterior a `fecha_expiracion`.
- Al firmar exitosamente se registra `fecha_firmado`, la IP del firmante (`request.getRemoteAddr()`) y se marca `utilizado = true`.
- Al firmar exitosamente el estado del proveedor se actualiza automáticamente al id 5 (Firmado).

6. Manejo de Requerimientos No Funcionales

En esta sección se describen los requerimientos no funcionales que deben soportarse alrededor de los servicios:

Aspecto no funcional	Requerimiento	Detalle del requerimiento
Invocación	Tipo de invocación	Síncrona únicamente. No existen colas ni reprocesamiento en background para estos tres servicios.
Invocación	Protocolo	REST sobre HTTP, formato JSON.
Seguridad	CORS	Configurado en CorsConfig; no se evidencia autenticación por token (JWT/OAuth) en los controladores revisados — la autorización por rol se maneja en el Frontend.
Calidad de Servicio	Tiempo de respuesta	No se define un SLA formal; registroCompleto y create de validación final son operaciones transaccionales sobre múltiples tablas.
Transacciones	Atomicidad	registroCompleto (Proveedor) y create (ValidacionFinal) están anotados con @Transactional; el resto de operaciones CRUD son de una sola escritura.

7. Manejo de Requerimientos Funcionales

7.1 Configuración en base de datos

Motor de base de datos: MySQL

Esquema: parere_grc

Nota: proveedor, estado_proveedor, validacion y campo_validacion están definidas en el script versionado Frontend/db/create_tables.sql. Las tablas validacion_final y firma_token no existen en ese script: se generan y actualizan automáticamente por Hibernate (spring.jpa.hibernate.ddl-auto=update) a partir de las entidades ValidacionFinalEntity y FirmaTokenEntity.

7.1.1 Tabla proveedor

Registro maestro de cada proveedor y su información general, financiera y de debida diligencia.

Columna	Descripción
id	Identificador único del proveedor (PK, autoincremental).
id_usuario	Usuario del portal de autogestión asociado al proveedor.
id_tipo_identificacion / numero_identificacion / digito_verificacion	Identificación tributaria o personal del proveedor.
razon_social / nombres / apellidos	Nombre según sea persona jurídica o natural (id_tipo_persona).
telefono_principal / correo_principal	Datos de contacto principal.
descripcion / ciu / pagina_web	Categoría, código CIU y sitio web de la empresa.
banco_referencia / tipo_cuenta / num_cuenta	Datos bancarios para pagos.
activos / pasivos / patrimonio / total_ingresos / total_gastos	Cifras financieras declaradas.
laft_p1 .. laft_p5	Respuestas booleanas al cuestionario SARLAFT/LAFT.
id_estado_proveedor	Estado actual del ciclo de vida (FK a estado_proveedor), por defecto 4.
fecha_aprobacion	Fecha en que el proveedor alcanzó un estado de aprobación (6 u 11).
activo	Bandera de borrado lógico. La entidad usa @SQLDelete + @SQLRestriction: un DELETE nunca borra la fila, solo pone activo = false.

7.1.2 Tabla validacion

Resultado de la comparación campo a campo entre el valor declarado en el formulario web y el valor extraído del documento soporte.

Columna	Descripción
id	Identificador único de la validación.
id_usuario / id_proveedor / id_campo_validacion	Quién valida, a qué proveedor y qué campo del catálogo campo_validacion.
valor_web / valor_documento	Valores comparados.
resultado_validacion	TRUE si coinciden, FALSE si no.
id_validacion_final	FK que enlaza esta validación atómica con la decisión consolidada (validacion_final) cuando aplica.
activo	Bandera de borrado lógico / vigencia del registro.

7.1.3 Tabla validacion_final (gestionada por Hibernate)

Bitácora unificada de decisiones de validación por rol (Compras, Riesgos, Oficial de Cumplimiento).
Reemplaza a la extinta tabla evaluacion_riesgos.

Columna	Descripción
id_validacion_final	Identificador único de la decisión.
id_proveedor	Proveedor evaluado.
comentario_final	Observación consolidada del evaluador.
estado_validacion	Código de decisión, usado también como id de estado a aplicar al proveedor.
creado_por / fecha_creado	Usuario y fecha de la decisión.
modificado_por / fecha_modificado	Auditoría de modificación (no expuesta aún por un endpoint de update).
activo	Bandera de vigencia del registro.

7.1.4 Tabla estado_proveedor

Catálogo administrable de estados del ciclo de vida del proveedor.

Columna	Descripción
id	Identificador único del estado (PK).
estado	Nombre corto y único del estado.
descripcion	Descripción del significado del estado.
activo	Bandera de borrado lógico.

7.1.5 Tabla campo_validacion

Catálogo de campos que requieren validación cruzada según el tipo de documento.

Columna	Descripción
id_campo_validacion	Identificador único del campo.
id_tipo_documento	Tipo de documento al que aplica (FK a tipo_documento).
campo	Nombre del campo a validar (ej. NIT, Dirección, Representante Legal).
obligatorio	Indica si el campo es de diligenciamiento obligatorio.
activo	Bandera de borrado lógico.

7.1.6 Tabla firma_token (gestionada por Hibernate)

Controla el ciclo de vida de cada enlace de firma digital enviado a un proveedor.

Columna	Descripción
id_token	Identificador único del token (PK).
id_proveedor	Proveedor al que pertenece la firma solicitada.
token	Cadena UUID única que viaja en la URL del formulario de firma.
fecha_expiracion	Momento a partir del cual el token deja de ser válido (creación + 24 horas).
utilizado	TRUE si el token ya fue consumido (firmado o invalidado por una solicitud posterior).
fecha_firmado / ip_firma	Auditoría de la aceptación digital: cuándo y desde qué IP se firmó.

Restricción observada: el script insert_data.sql solo siembra tres estados (ACTIVO, INACTIVO, SUSPENDIDO), mientras que la lógica de negocio referencia los identificadores 4 (por defecto), 5 (Firmado), 6 (Aprobado), 10 (Revisado con novedad) y 11 (Revisado sin novedad). En la práctica, el catálogo completo de estados debe mantenerse actualizado manualmente a través del CRUD administrativo /estado_proveedor para que los tres servicios documentados operen correctamente.

7.2 Llamadas del servicio

Los tres servicios se invocan de forma exclusivamente síncrona: el cliente (Frontend) espera la respuesta HTTP antes de continuar. No existe un modo de invocación asíncrono (equivalente a invocationMode en otros servicios del ecosistema) ni un timer de reprocesamiento de transacciones fallidas.

El flujo de Validación Final (5.2) requiere dos llamadas secuenciales desde el Frontend — POST /validacion-final seguido de PATCH /proveedores/{id}/estado — que no están envueltas en una única transacción distribuida: si la segunda llamada falla, la decisión queda registrada en validacion_final pero el estado del proveedor no se actualiza.

7.3 Manejo de errores

El backend no implementa un @ControllerAdvice ni un catálogo propio de códigos de error (a diferencia de otros servicios del ecosistema que usan códigos como NTF07-NTF11). Las validaciones de negocio (registro inexistente, token inválido, expirado o ya usado) se señalizan lanzando RuntimeException con un mensaje descriptivo, que Spring Boot traduce en una respuesta HTTP 500 con el formato de error por defecto:

```
{
  "timestamp": "2026-08-23T15:40:12.000+00:00",
```

```
"status": 500,  
"error": "Internal Server Error",  
"message": "El enlace de firma ha expirado",  
"path": "/firmas/validar"  
}
```

Origen	Mensaje	Causa
ValidacionService / ProveedorService	"El registro no existe"	Se consulta, actualiza o elimina un id que no existe en la tabla correspondiente.
FirmaTokenService	"El proveedor con ID {id} no existe"	Se solicita una firma para un idProveedor inexistente.
FirmaTokenService	"El token de firma no existe"	El token recibido no está registrado en firma_token.
FirmaTokenService	"El enlace de firma ya ha sido utilizado"	El token ya fue consumido en una firma o invalidación previa.
FirmaTokenService	"El enlace de firma ha expirado"	La fecha actual supera fecha_expiracion (24 horas desde la creación).

Se sugiere, como mejora futura, introducir un manejador global de excepciones que traduzca estos casos de negocio en códigos HTTP semánticamente correctos (404 para "no existe", 409/410 para tokens usados o expirados) en lugar de un 500 genérico.